

EE332

Digital System Design

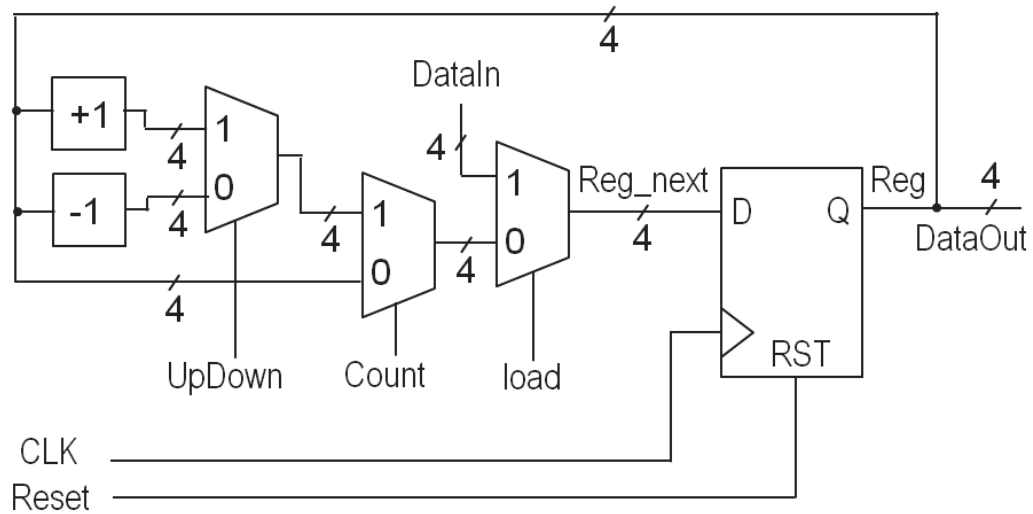
Dr. Yi Zeng
Assistant Professor,
Department of Electrical and Electronic Engineering
Office: S236, South Tower of COE
Email: zengyi@sustech.edu.cn

6.2.5 Synchronous counter

```
-- 4-bit synchronous counter
library IEEE;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use ieee.std_logic_unsigned.all;

entity counter is
port (CLK, RESET: in std_logic;
      load, Count, UpDown: in std_logic;
      DataIn: in std_logic_vector(3 downto 0);
      DataOut: out std_logic_vector(3 downto 0));
end entity counter;

architecture two-seg of counter is
signal Reg, Reg_Next : std_logic_vector (3 downto 0);
```



Conceptual Diagram

- 1、先画flip-flop
- 2、优先级：load > count > updown

```

begin
process (CLK, RESET) is
begin

```

```

if RESET = '1' then

```

```

    Reg <= "0000";

```

```

elseif CLK'event and CLK='1' then

```

```

    Reg <= Reg_Next;

```

```

end if;

```

```

end process;

```

```

-- next-state logic (并发赋值语句)

```

```

Reg_next <= DataIn when load = '1' else

```

```

    (Reg+1) when (Count='1' and UpDown = '1') else

```

```

    (Reg-1) when (Count='1' and UpDown='0') else

```

```

    Reg;

```

```

-- Output logic

```

```

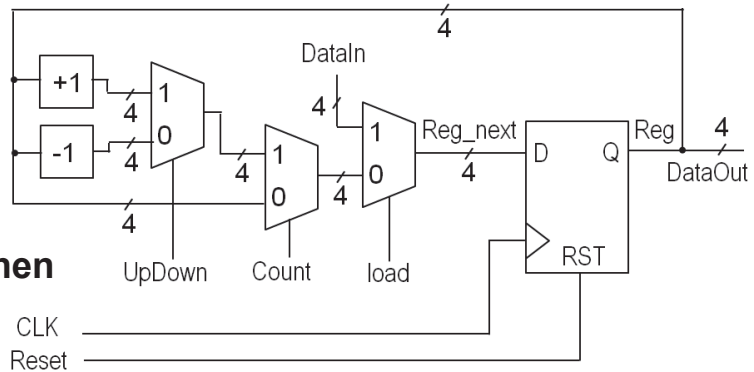
DataOut <= Reg;

```

```

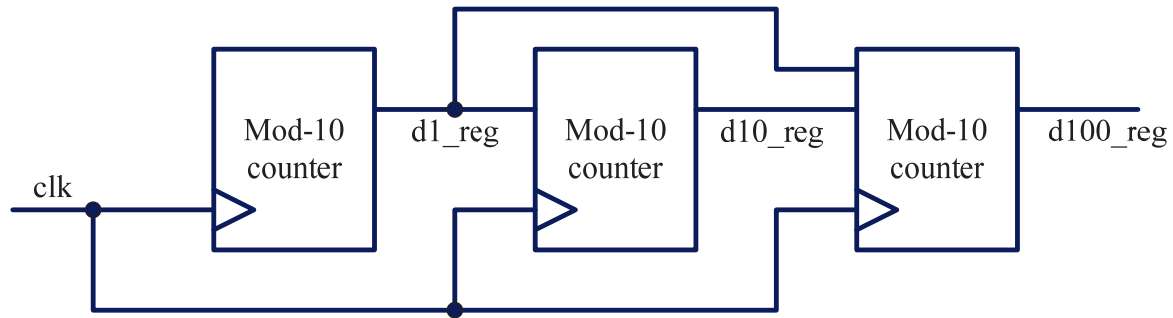
end architecture two-seg;

```



Decimal counter

- A decimal counter circulates the patterns in binary-coded decimal (BCD) format.
- The BCD code use 4 bits to represent a decimal number.



Three-digit decimal counter using conditional concurrent statements

```
library IEEE;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use ieee.std_logic_unsigned.all;

entity decimal_counter is
port (CLK, RESET: in std_logic;
      d1, d10, d100: out std_logic_vector(3 downto 0));
end entity decimal_ounter;

architecture concurrent_arch of decimal_ounter is
signal d1_reg, d10_reg, d100_reg: std_logic_vector (3 downto 0);
signal d1_next, d10_next, d100_next: std_logic_vector (3 downto 0);

begin
```

-- register

process (CLK, RESET) is
begin

if RESET = '1' then

 d1_reg <="0000";

 d10_reg <="0000";

 d100_reg <="0000";

elsif CLK'event and CLK='1' then

 d1_reg <= d1_next;

 d10_reg <= d10_next;

 d100_reg <= d100_next;

end if;

end process;

-- next-state logic

d1_next <= "0000" when d1_reg = 9 else d1_reg+1;

d10_next <= "0000" when (d1_reg = 9 and d10_reg = 9) else

 d10_reg+1 when d1_reg = 9 else d10_reg;

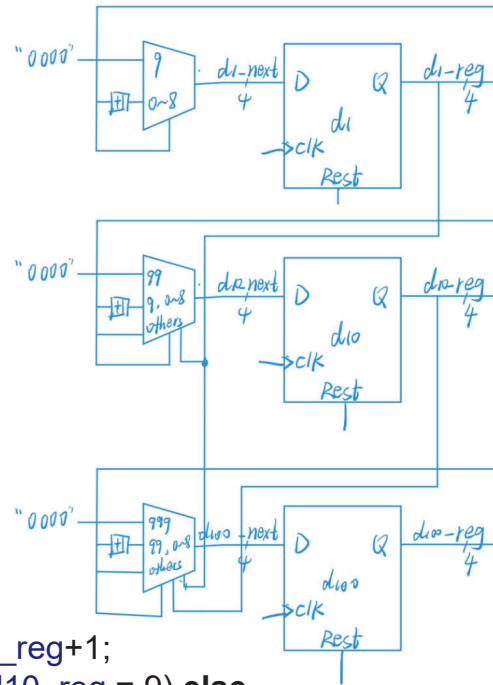
d100_next <= "0000" when (d1_reg=9 and d10_reg=9 and d100_reg=9) else

 d100_reg+1 when (d1_reg=9 and d10_reg=9) else d100_reg;

-- Output logic

d1 <= d1_reg; d10 <= d10_reg; d100 <= d100_reg;

end architecture concurrent arch



Three-digit decimal counter using a nested if statement

```
architecture if_arch of decimal_ounter is
    signal d1_reg, d10_reg, d100_reg: std_logic_vector (3 downto 0);
    signal d1_next, d10_next, d100_next: std_logic_vector (3 downto 0);
begin
    process (CLK, RESET) is
    begin
        if RESET = '1' then
            d1_reg <="0000";
            d10_reg <="0000";
            d100_reg <="0000";
        elsif CLK'event and CLK='1' then
            d1_reg <= d1_next;
            d10_reg <= d10_next;
            d100_reg <= d100_next;
        end if;
    end process;
```


-- next-state logic

```
process (d1_reg, d10_reg, d100_reg)
begin
```

```
    d10_next <= d10_reg;
    d100_next <= d100_reg;
    if d1_reg /= 9 then
```

!= 不等于

```
        d1_next = d1_reg+1;
```

```
    else -- reach 9
```

```
        d1_next = "0000";
```

```
        if d10_reg /= 9 then
```

```
            d10_next <= d10_reg + 1;
```

```
        else -- reach 99
```

```
            d10_next <= "0000";
```

```
            if d100_reg /= 9 then
```

```
                d100_next <= d100_reg + 1;
```

```
            else -- reach 999
```

```
                d100_next <= "0000";
```

```
            end if;
```

```
        end if;
```

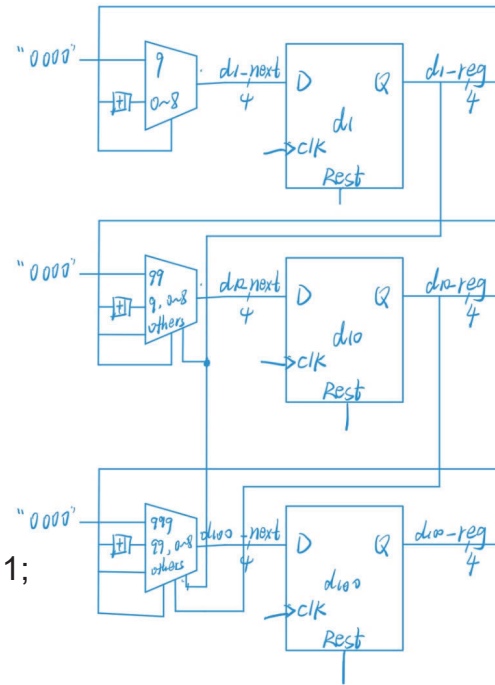
```
    end if;
```

```
end process;
```

-- Output logic

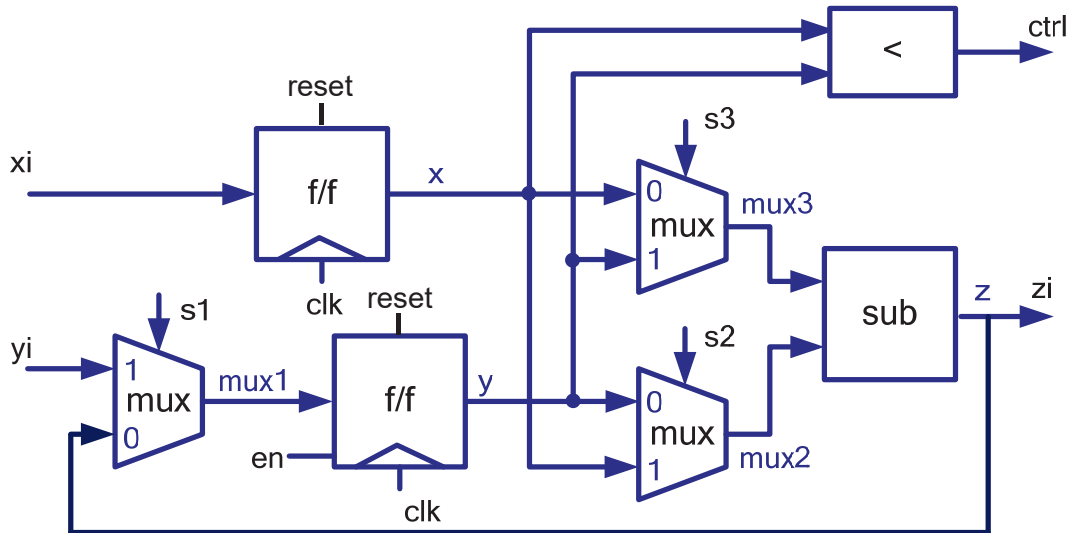
```
d1 <= d1_reg; d10 <= d10_reg; d100 <= d100_reg;
```

```
end architecture if_arch;
```



6.3 Netlist of RTL components

- A data path usually consists of a netlist of RTL components such as function units, multiplexers, comparators, registers, etc.



```

signal clk, en, s1, s2, s3 : s
signal xi, yi, zi : std_logic_vector(xi)
signal ctrl : boolean;

```

.....

```

architecture two_seg of da

```

```

signal x, y, z, x_next, y_next, yi

```

```

begin

```

```

process (clk) is

```

```

begin

```

```

if reset = '1' then

```

```

    x <= "00000000";

```

```

    y <= "00000000";

```

```

elsif (clk'event and clk='1') then -- registers

```

```

    x <= x_next;

```

```

    y <= y_next;

```

```

end if;

```

```

end process;

```

```

ctrl <= (x < y); -- comparator

```

```

x_next <= xi;

```

```

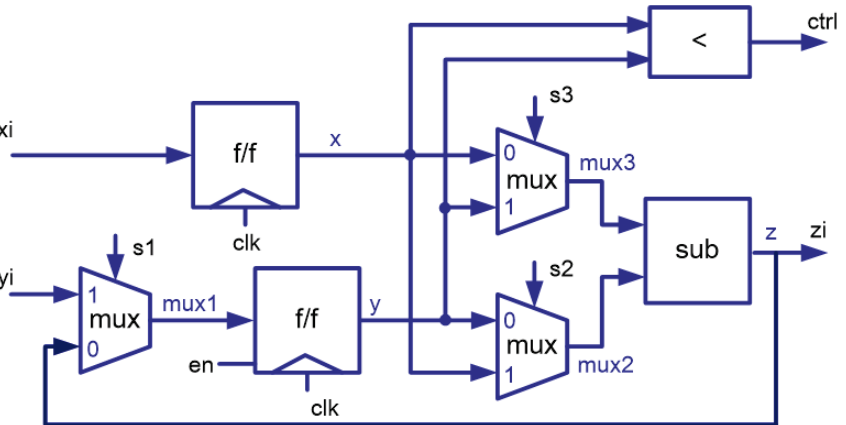
y_next <= mux1 when en = '1' else

```

```

    yi;

```



```

mux1 <= z when s1 = '0' else
    yi; -- multiplexer
mux2 <= y when s2 = '0' else
    x; -- multiplexer
mux3 <= x when s3 = '0' else
    y; -- multiplexer
z <= mux3 - mux2; -- subtractor
zi <= z;
end architecture two_seg;

```

6.4 Test benches for sequential system

- All synchronous system require a system clock signal.
- A reset signal is required. The reset signal is asserted at power on to place the sequential system in its initial state.

6.4.1 Generating a system clock

- 50% duty cycle clock

```
clock_gen: process  
constant period : time := 100 ns;  
begin  
    clk <= '0';  
    wait for period/2;  
    clk <= '1';  
    wait for period/2;  
end process;
```

6.4.2 Generating the system reset

- The reset signal typically
 - starts in its asserted state at power on,
 - remains in that state for a specified period of time, then
 - changes to its unasserted state, and
 - remains there for as long as power continues to be applied to the system.

- The duration of the assertion of the reset signal is specified as
 - either a fixed time
 - or some multiple of the clock's period and is synchronized to the system clock

```
reset <= '1', '0' after 160 ns;
```

```
reset_process : process
begin
    reset <= '1';
    for i in 1 to 2 loop
        wait until clk = '1';
    end loop;
    reset <= '0';
    wait;
end process;
```

6.4.3 Synchronizing stimulus generation and monitoring

monitor : process

 constant n : integer := 1000;

 variable number : integer range 0 to 999 :=0;

begin 仿真写testbench时可以用variable

 wait until reset <= '0';

 wait for 1 ns;



 for i in 0 to n loop

 number := to_integer(unsigned(d100))*100+to_integer(unsigned(d10))*10
 + to_integer(unsigned(d1));

 assert number = i mod n integer'image(): 将整数转换成字符串

 report "count of " & integer'image(i mod n) & " failed"

 severity error;

 wait until clk = '1';

 wait for 1 ns;

end loop;

wait;

end process;

-- register

process (CLK, RESET) is
begin

if RESET = '1' then

 d1_reg <="0000";

 d10_reg <="0000";

 d100_reg <="0000";

elsif CLK'event and CLK='1' then

 d1_reg <= d1_next;

 d10_reg <= d10_next;

 d100_reg <= d100_next;

end if;

end process;

-- next-state logic

d1_next <= "0000" when d1_reg = 9 else d1_reg+1;

d10_next <= "0000" when (d1_reg = 9 and d10_reg = 9) else

 d10_reg+1 when d1_reg = 9 else d10_reg;

d100_next <= "0000" when (d1_reg=9 and d10_reg=9 and d100_reg=9) else

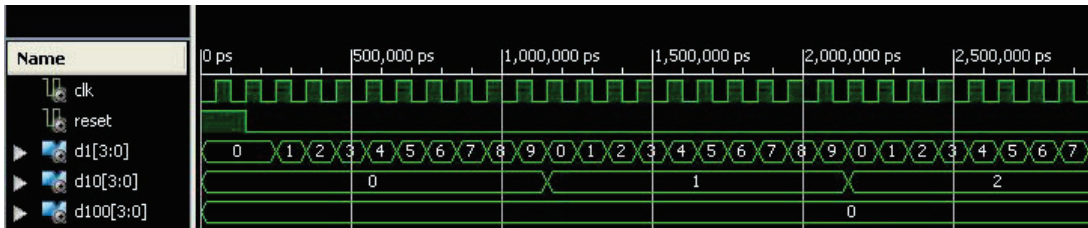
 d100_reg+1 when (d1_reg=9 and d10_reg=9) else d100_reg;

-- Output logic

d1 <= d1_reg; d10 <= d10_reg; d100 <= d100_reg;

end architecture concurrent_arch;

Waveforms of clk and reset



```
use ieee.numeric_std.all;  
signal x : std_logic_vector(3 downto 0); -- vector with element std_logic  
signal y : unsigned(3 downto 0); -- vector with element std_logic  
signal z : integer range 0 to 15;
```

conversion between std_logic_vector, signed, unsigned

```
x <= y; -- illegal assignment, type conflict
```

```
y <= x; -- illegal assignment, type conflict
```

```
x <= std_logic_vector(y); -- legal assignment
```

```
y <= unsigned(x); -- legal assignment
```

conversion between signed, unsigned, integer

```
z <= to_integer(y); -- legal assignment
```

```
z <= to_integer(unsigned(x)); -- legal assignment
```

```
y <= to_unsigned(z, 4); -- legal assignment
```

```
x <= std_logic_vector(to_unsigned(z, 4)); -- legal assignment
```